

Angular Answers

Hands-On 1 — Environment Setup, Project Structure & First Component

Install Angular CLI, scaffold the project, explore files, create core components.

Task 1 — Scaffold and Explore the Angular Project

Step 1–3: Create project and serve

```
# Install Angular CLI globally
npm install -g @angular/cli

# Create the project
ng new student-course-portal --routing --style=css

# Enter the folder and serve
cd student-course-portal ng
serve

✓ Packages installed successfully.
✓ Angular project created.
** Angular Live Development Server is listening on localhost:4200 **
Application bundle generation complete.
→ Open http://localhost:4200 in the browser.
    Default Angular welcome page is visible.
```

Step 2: notes.txt — File explanations

```
# notes.txt

angular.json      - Workspace configuration: build targets, serve options, budgets, assets.
tsconfig.json     - Root TypeScript compiler options shared by all projects in the workspace.
tsconfig.app.json - TypeScript config specific to the application build (extends tsconfig.json).
package.json      - npm metadata: project dependencies, devDependencies, and npm scripts.
src/main.ts       - Application entry point; bootstraps the root component (or AppModule).
src/app/app.config.ts - (Standalone mode) Provides root-level services and router configuration.
src/app/app.component.ts - Root component; the first component loaded after bootstrap.
src/index.html    - Shell HTML file; Angular injects <app-root> and compiled bundles here.
```

Step 4–5: Build and inspect budgets

```
ng build # Inspect dist/student-
course-portal/
#   main.js    compiled application code
#   polyfills.js, styles.css, index.html
```

In angular.json → architect → build → configurations → production → budgets: maximumWarning: '500kb' — warn if bundle exceeds this size. maximumError: '1mb' — fail the build if bundle exceeds this size.

Output after ng build:

```
dist/student-course-portal/
index.html
  main-XXXXXXXXX.js      compiled application code
polyfills-XXXXXXXXX.js   styles-XXXXXXXXX.css Build
complete.
```

Task 2 — Create and Organise Components

Step 6: Generate components

```
ng generate component components/header
ng generate component pages/home ng
generate component pages/course-list ng
generate component pages/student-profile
```

```
CREATE src/app/components/header/header.component.ts
CREATE src/app/components/header/header.component.html
CREATE src/app/components/header/header.component.css
CREATE src/app/components/header/header.component.spec.ts
(Same 4 files created for home, course-list, student-profile)
```

Step 7: header.component.html

```
<nav>  <span class="portal-name">Student Course
Portal</span>
  <a routerLink="/">Home</a>
  <a routerLink="/courses">Courses</a>
  <a routerLink="/profile">Profile</a>
</nav>
```

Step 8: home.component.html

```
<h1>Welcome to the Student Course Portal</h1> <p>Browse courses,
manage enrolments and track your progress.</p>

<div class="stats-row">
  <div class="stat"><strong>Courses Available:</strong> 12</div>
  <div class="stat"><strong>Enrolled:</strong> 3</div>
  <div class="stat"><strong>GPA:</strong> 3.8</div>
</div>
```

Step 9: app.component.html

```
<app-header></app-header> <router-outlet></router-outlet> ng serve - no errors.
Browser shows the header nav bar and the home page welcome text with the stats row.
```

Hands-On 2 — Data Binding, Lifecycle Hooks & Component Communication

Task 1 — All Four Binding Types

home.component.ts

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({  selector: 'app-home',
  standalone: true,  imports:
    [FormsModule],  templateUrl:
    './home.component.html' })
export class HomeComponent {  portalName
= 'Student Course Portal';
isPortalActive = true;  message      =
  '';  searchTerm  = '';

  onEnrollClick() {
    this.message = 'Enrollment opened!';
  }
}
```

home.component.html

```
<!-- Interpolation -->
<h1>{{ portalName }}</h1>

<!-- Property binding (one-way: component → DOM) -->
<button [disabled]="!isPortalActive" (click)="onEnrollClick()">Enroll Now</button>
<p *ngIf="message">{{ message }}</p>

<!-- Two-way binding (DOM ↔ component) -->
<input [(ngModel)]="searchTerm" placeholder="Search courses..." />
<p>Searching for: {{ searchTerm }}</p>

<!--
  [property] = one-way: component → DOM. DOM changes do NOT update the component.
  [(ngModel)] = two-way: Angular syncs both directions automatically.
-->

Output:
  Typing in the search box instantly updates "Searching for: <typed text>"

Clicking "Enroll Now" shows "Enrollment opened!" below the button.
```

Task 2 — Lifecycle Hooks *home.component.ts* — *ngOnInit* &

ngOnDestroy

```
import { Component, OnInit, OnDestroy } from '@angular/core';

export class HomeComponent implements OnInit, OnDestroy {
  courseCount = 0;

  ngOnInit() {    // Simulate
    fetching course count
    this.courseCount = 12;
    console.log('HomeComponent initialised — courses loaded');
  }

  ngOnDestroy() {
    console.log('HomeComponent destroyed');
  }
}
```

Generate CourseCardComponent and implement ngOnChanges

```
ng generate component components/course-card
// course-card.component.ts
import { Component, Input, OnChanges, SimpleChanges } from '@angular/core';

@Component({ selector: 'app-course-card', standalone: true, templateUrl: '...' })
export class CourseCardComponent implements OnChanges {   @Input() course: any;

    ngOnChanges(changes: SimpleChanges) {
    if (changes['course']) {
    console.log('course changed:',
    changes['course'].previousValue,
        '→',
        changes['course'].currentValue);
    }
    }
}

Console output on page load:   HomeComponent
initialised – courses loaded

    course changed: undefined → {id:1, name:'Data Structures', ...}
course changed: undefined → {id:2, name:'Algorithms', ...}
course changed: undefined → {id:3, name:'Web Development', ...}

On navigation away:
    HomeComponent destroyed
```

Task 3 — @Input and @Output course-

card.component.ts

```
import { Component, Input, Output, EventEmitter } from '@angular/core';

export class CourseCardComponent {   @Input() course!: { id: number; name:
string; code: string; credits: number };   @Output() enrollRequested = new
EventEmitter<number>(); } course-card.component.html
<div class="card">
    <h3>{{ course.name }}</h3>
    <p>Code: {{ course.code }}</p>
    <p>Credits: {{ course.credits }}</p>   <button
(click)="enrollRequested.emit(course.id)">Enroll</button>
</div> course-
```

list.component.ts

```
export class CourseListComponent {   courses = [       { id: 1, name:
'Data Structures',       code: 'CS101', credits: 4 },       { id: 2, name:
'Algorithms',       code: 'CS102', credits: 3 },
        { id: 3, name: 'Web Development',       code: 'CS103', credits: 3 },
        { id: 4, name: 'Database Systems',       code: 'CS104', credits: 4 },
{ id: 5, name: 'Operating Systems',       code: 'CS105', credits: 3 },   ];
    selectedCourseId: number | null = null;

    onEnroll(courseId: number) {
        console.log('Enrolling in course: ' + courseId);
        this.selectedCourseId = courseId;   }
} course-
```

list.component.html

```
<app-course-card
    *ngFor="let c of courses"
    [course]="c"
    (enrollRequested)="onEnroll($event)">
</app-course-card>
<p *ngIf="selectedCourseId">Selected course ID: {{ selectedCourseId }}</p>
```

Clicking "Enroll" on a card:

Console → "Enrolling in course: 2"

Page shows → "Selected course ID: 2"

Hands-On 3 — Directives & Pipes

Task 1 — Structural Directives *course-*

list.component.ts — *ngOnInit* with *loading*

```
isLoading = true;
courses: Course[] = [ /* 5 course objects with gradeStatus */ ];

ngOnInit() {
  setTimeout(() => { this.isLoading = false; }, 1500);
}

trackByCourseId(index: number, course: Course): number { return course.id;
// trackBy avoids re-rendering every DOM node on array change; // Angular
only patches items whose identity (course.id) actually changed.
} course-
```

list.component.html

```
<p *ngIf="isLoading">Loading courses...</p>

<ng-container *ngIf="!isLoading"> <ng-container
*ngIf="courses.length > 0; else noCourses">
  <app-course-card *ngFor="let course of courses; let i = index;
trackBy: trackByCourseId"
  [course]="course">
    </app-course-card>
</ng-container>
<ng-template #noCourses>
<p>No courses available.</p>
</ng-template>
</ng-container> course-card.component.html —
```

**ngSwitch* badge

```
<ng-container [ngSwitch]="course.gradeStatus">
  <span *ngSwitchCase="'passed'" class="badge badge--green">Passed</span>
  <span *ngSwitchCase="'failed'" class="badge badge--red">Failed</span>
  <span *ngSwitchDefault class="badge badge--grey">Pending</span>
</ng-container>
```

Output:

First 1.5 seconds — "Loading courses..." is visible.

After 1.5 s — course cards appear, each with the correct colour-coded badge.

Task 2 — Attribute Directives: *ngClass* and *ngStyle* *course-*

card.component.ts — *getter* for *classes*

```
isExpanded = false;

// Getter keeps template clean — logic stays in TypeScript, not inline HTML
get cardClasses() { return { 'card--enrolled': this.course.enrolled,
  'card--full': this.course.credits >= 4,
  'expanded': this.isExpanded
};
} course-card.component.html — ngClass &
```

ngStyle

```
<div [ngClass]="cardClasses" [ngStyle]="{ 'border-left-
color': course.gradeStatus === 'passed' ? 'green' :
course.gradeStatus === 'failed' ? 'red' : 'grey' }">

  <h3>{{ course.name }}</h3>
  <!-- ... other fields ... -->
```

```

    <button (click)="isExpanded = !isExpanded">      {{
isExpanded ? 'Hide Details' : 'Show Details' }}
    </button>
</div>

Enrolled cards show a highlighted border via .card--enrolled.
High-credit cards (credits >= 4) get .card--full styling.
Clicking "Show Details" toggles the .expanded class, expanding card height.

```

Task 3 — Custom Directive and Custom Pipe

Generate files

```

ng generate directive directives/highlight
ng generate pipe pipes/credit-label

```

highlight.directive.ts

```

import { Directive, HostListener, ElementRef, Input } from '@angular/core';

@Directive({ selector: '[appHighlight]', standalone: true })
export class HighlightDirective {   @Input() appHighlight =
'yellow';

  constructor(private el: ElementRef) {}

  @HostListener('mouseenter') onEnter() {
    this.el.nativeElement.style.backgroundColor = this.appHighlight;
  }   @HostListener('mouseleave') onLeave() {
this.el.nativeElement.style.backgroundColor = '';   }
} credit-

```

label.pipe.ts

```

import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'creditLabel', standalone: true }) export
class CreditLabelPipe implements PipeTransform {
  transform(value: number | null): string {    if (!value
|| value <= 0) return 'No Credits';    return value ===
1 ? '1 Credit' : `${value} Credits`;    }
}

```

Usage in template

```

<!-- Apply directive with optional custom colour --> <app-course-
card appHighlight="lightblue" ...></app-course-card>

<!-- Use pipe -->
<p>{{ course.credits | creditLabel }}</p>

<!-- Output:  1 → "1 Credit"    3 → "3 Credits"    null → "No Credits" -->

Hovering over a course card turns its background lightblue.
{{ 3 | creditLabel }} renders "3 Credits".
{{ 1 | creditLabel }} renders "1 Credit".
{{ null | creditLabel }} renders "No Credits".

```

Hands-On 4 — Template-Driven Forms & Validation

Task 1 — Build the Enrollment Request Form

Generate component and add route

```

ng generate component pages/enrollment-form

# app-routing.module.ts
{ path: 'enroll', component: EnrollmentFormComponent } enrollment-

```

form.component.ts

```
import { Component } from '@angular/core'; import {
FormsModule, NgForm } from '@angular/forms';
```

```
@Component({
  selector: 'app-enrollment-form',
  standalone: true, imports:
[FormsModule],
  templateUrl: './enrollment-form.component.html',
  styleUrls: ['./enrollment-form.component.css'] })
export class EnrollmentFormComponent {
  submitted = false;

  onSubmit(form: NgForm) {
    console.log('form.value  =', form.value);
    console.log('form.valid   =', form.valid);
    if (form.valid) {      this.submitted = true;
    }
  }
} enrollment-
```

form.component.html

```
<form #enrollForm="ngForm" (ngSubmit)="onSubmit(enrollForm)">

  <label>Student Name      <input name="studentName"
[(ngModel)]="model.studentName"          required minlength="3"
#nameCtrl="ngModel" />      <span *ngIf="nameCtrl.touched &&
nameCtrl.errors?.['required']">      Name is required.
    </span>      <span *ngIf="nameCtrl.touched &&
nameCtrl.errors?.['minlength']">      Name must be at least 3
characters.
    </span>
  </label>

  <label>Email      <input name="studentEmail" type="email" [(ngModel)]="model.studentEmail"
required email #emailCtrl="ngModel" />      <span *ngIf="emailCtrl.touched &&
emailCtrl.errors?.['required']">Email is required.</span>      <span *ngIf="emailCtrl.touched &&
emailCtrl.errors?.['email']">Enter a valid email.</span>
  </label>

  <label>Course ID      <input name="courseId" type="number"
[(ngModel)]="model.courseId" required />
  </label>

  <label>Preferred Semester      <select name="preferredSemester"
[(ngModel)]="model.preferredSemester" required>
    <option value="Odd">Odd</option>
<option value="Even">Even</option>
  </select>
  </label>

  <label>
    <input name="agreeToTerms" type="checkbox" [(ngModel)]="model.agreeToTerms" required />
    I agree to the terms
  </label>

  <button type="submit" [disabled]="enrollForm.invalid">Submit</button>
<button type="button" (click)="enrollForm.resetForm()">Reset</button>

  <div *ngIf="submitted" class="success">
    Enrollment request submitted successfully!
  </div>
</form> enrollment-
```

form.component.css

```
input.ng-invalid.ng-touched,
select.ng-invalid.ng-touched { border-color: red; }
```

```
input.ng-valid.ng-touched,  
select.ng-valid.ng-touched { border-color: green; }  
  
.success { color: green; margin-top: 10px; font-weight: bold; }
```

Output:

Submitting empty form → Submit button disabled.

Touching Name and leaving blank → red border + "Name is required."

Entering "Jo" → "Name must be at least 3 characters."

Valid input → green border.

All fields valid + submit → Console logs form.value object.

Success message appears: "Enrollment request submitted successfully!"

Hands-On 5 — Reactive Forms, FormBuilder, FormArray & Custom Validators

Task 1 — Reactive Form with FormBuilder

Generate component

```
ng generate component pages/reactive-enrollment-form # Route: { path: 'enroll-
reactive', component: ReactiveEnrollmentFormComponent } reactive-enrollment-
form.component.ts

import { Component, OnInit } from '@angular/core';
import { ReactiveFormsModule, FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-reactive-enrollment-form',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './reactive-enrollment-form.component.html'
})
export class ReactiveEnrollmentFormComponent implements OnInit {
  enrollForm!: FormGroup;

  constructor(private fb: FormBuilder) {}

  ngOnInit() {
    this.enrollForm = this.fb.group({
      studentName: ['', [Validators.required, Validators.minLength(3)]],
      studentEmail: ['', [Validators.required, Validators.email]],
      courseId: [null, Validators.required], preferredSemester:
      ['Odd', Validators.required], agreeToTerms: [false,
      Validators.requiredTrue], additionalCourses: this.fb.array([]) });
  }

  onSubmit() { // .value excludes disabled controls; .getRawValue()
    includes all controls console.log('value: ',
    this.enrollForm.value); console.log('getRawValue(): ',
    this.enrollForm.getRawValue()); }

  get additionalCourses() { // Typed getter avoids casting in the
    template — keeps templates clean return
    this.enrollForm.get('additionalCourses') as FormArray; }

  addCourse() {
    this.additionalCourses.push(this.fb.control('', Validators.required));
  }

  removeCourse(i: number) { this.additionalCourses.removeAt(i); }
} reactive-enrollment-form.component.html

<form [formGroup]="enrollForm" (ngSubmit)="onSubmit()">

  <input formControlName="studentName" placeholder="Student Name" />
  <input formControlName="studentEmail" placeholder="Email" type="email" />
  <input formControlName="courseId" placeholder="Course ID" />

  <span *ngIf='enrollForm.get("courseId").errors?.["noCourseCode"]'>
    Course code starting with XX is not allowed.
  </span> <span
  *ngIf='enrollForm.get("studentEmail").errors?.["emailTaken"]'>
    Email is already taken (checked after 800 ms).
  </span>
  <select formControlName="preferredSemester">
    <option value="Odd">Odd</option>
  <option value="Even">Even</option>
  </select> <input formControlName="agreeToTerms"
  type="checkbox" />
```

```

<!-- FormArray -->
<div formArrayName="additionalCourses">
  <div *ngFor="let ctrl of additionalCourses.controls; let i = index">
    <input [formControlName]="i" placeholder="Additional course" />
    <button type="button" (click)="removeCourse(i)">Remove</button>
  </div>
</div>
<button type="button" (click)="addCourse()">+ Add Another Course</button>

<button type="submit" [disabled]="enrollForm.invalid">Submit</button>
</form>

```

Task 2 — Custom Validators *validators/no-course-*

code.validator.ts

```

import { AbstractControl, ValidationErrors } from '@angular/forms';

export function noCourseCode(control: AbstractControl): ValidationErrors | null {
  const val = control.value as string;
  if (val && val.startsWith('XX')) return { noCourseCode: true };
return null; } validators/email-taken.validator.ts (async)
import { AbstractControl, ValidationErrors } from '@angular/forms';

export function simulateEmailCheck(control: AbstractControl):
Promise<ValidationErrors | null> { return new
Promise(resolve => { setTimeout(() => {
  resolve(control.value?.includes('test@') ? { emailTaken: true } : null);
}, 800);
});
}

```

Apply validators in ngOnInit

```

courseId: [null, [Validators.required, noCourseCode]],
studentEmail: ['', [Validators.required, Validators.email], [simulateEmailCheck]],

Entering "XX101" in Course ID → "Course code starting with XX is not allowed."
Entering "test@example.com" → after 800 ms → "Email is already taken."
Clicking "+ Add Another Course" adds a new input row.
Clicking "Remove" deletes that row.
Submit logs form.value and getRawValue() to console.

```

Hands-On 6 — Services & Dependency Injection

Task 1 — Course Service

models/course.model.ts

```

export interface Course {
  id: number;
  name: string;
  code: string;
  credits: number;
  gradeStatus: 'passed' | 'failed' | 'pending';
  enrolled?: boolean; }

```

services/course.service.ts

```

import { Injectable } from '@angular/core';
import { Course } from '../models/course.model';

@Injectable({ providedIn: 'root' }) // singleton — one shared instance export class
CourseService { private courses: Course[] = [ { id:1, name:'Data Structures',
code:'CS101', credits:4, gradeStatus:'passed' },
  { id:2, name:'Algorithms', code:'CS102', credits:3, gradeStatus:'pending' },
  { id:3, name:'Web Development', code:'CS103', credits:3, gradeStatus:'failed' },

```

```

    { id:4, name:'Database Systems', code:'CS104', credits:4, gradeStatus:'passed' },
    { id:5, name:'Operating Systems', code:'CS105', credits:3, gradeStatus:'pending' },    ];

    getCourses(): Course[] { return this.courses; }
    getCourseById(id: number): Course | undefined {
    return this.courses.find(c => c.id === id);    }
    addCourse(course: Course): void { this.courses.push(course); }
}

```

Inject into components

```

// course-list.component.ts
constructor(private courseService: CourseService) {}
ngOnInit() { this.courses = this.courseService.getCourses(); }

// home.component.ts
constructor(private courseService: CourseService) {} get
courseCount() { return this.courseService.getCourses().length; }

Both HomeComponent and CourseListComponent read from the same service instance.
Adding a course via addCourse() from any component updates the count everywhere.

```

Task-2 nrollment Service & Hierarchical DI

services/enrollment.service.ts

```

import { Injectable } from '@angular/core';
import { CourseService } from '../course.service';
import { Course } from '../models/course.model';

@Injectable({ providedIn: 'root' }) export
class EnrollmentService {    private
enrolledCourseIds: number[] = [];

    constructor(private courseService: CourseService) {} // service-to-service injection

    enroll(courseId: number): void {
        if (!this.isEnrolled(courseId)) this.enrolledCourseIds.push(courseId);
    }
    unenroll(courseId: number): void {
        this.enrolledCourseIds = this.enrolledCourseIds.filter(id => id !== courseId);
    }
    isEnrolled(courseId: number): boolean {    return
this.enrolledCourseIds.includes(courseId);    }
    getEnrolledCourses(): Course[] {    return
this.enrolledCourseIds        .map(id =>
this.courseService.getCourseById(id))
.filter((c): c is Course => !!c);
    }
} course-card.component.ts — toggle

```

enroll

```

constructor(private enrollmentService: EnrollmentService) {}

get isEnrolled() { return this.enrollmentService.isEnrolled(this.course.id); }

onEnrollClick() {    this.isEnrolled        ?
this.enrollmentService.unenroll(this.course.id)        :
this.enrollmentService.enroll(this.course.id);
}

```

Component-level scoped service (NotificationComponent)

```

@Component({
    selector: 'app-notification',
    providers: [NotificationService],    // NEW instance per component – not the root singleton
    // ... })
export class NotificationComponent { ... }

```

```

/*  Component-level providers create a separate service instance scoped to this
    component  and its children. Useful for per-instance state (e.g. per-dialog
    notifications).  It does NOT share state with the root-level NotificationService
    instance.
*/

```

Clicking "Enroll" → button label changes to "Unenroll".

Clicking again → reverts to "Enroll".

StudentProfileComponent shows the enrolled courses list.

Hands-On 7 — Angular Routing: Guards, Lazy Loading & Route Data

Task 1 — Route Configuration, Parameters and Nested Routes

app.routes.ts

```

import { Routes } from '@angular/router';
import { HomeComponent } from './pages/home/home.component'; import {
CourseListComponent } from './pages/course-list/course-list.component'; import {
CourseDetailComponent } from './pages/course-detail/course-detail.component'; import {
StudentProfileComponent } from './pages/student-profile/student-profile.component'; import {
NotFoundComponent } from './pages/not-found/not-found.component'; import {
CoursesLayoutComponent } from './pages/courses-layout/courses-layout.component';

export const routes: Routes = [  { path: '',
component: HomeComponent },  { path: 'profile',
component: StudentProfileComponent,    canActivate:
[AuthGuard] },  {
    path: 'courses',
    component: CoursesLayoutComponent,    children: [
{ path: '',    component: CourseListComponent },    {
path: ':id', component: CourseDetailComponent },
    ]
},  { path: 'enroll', loadChildren:
() =>
    import('./features/enrollment/enrollment.module').then(m => m.EnrollmentModule),
    canActivate: [AuthGuard] },  { path: '**', component: NotFoundComponent }  //
wildcard must be last
]; course-

```

detail.component.ts

```

import { ActivatedRoute } from '@angular/router'; import {
CourseService } from '../services/course.service'; import {
Course } from '../models/course.model';

export class CourseDetailComponent implements OnInit {
course: Course | undefined;

    constructor(private route: ActivatedRoute,
private courseService: CourseService) {}

    ngOnInit() {
        const id = Number(this.route.snapshot.paramMap.get('id'));
        this.course = this.courseService.getCourseById(id);    }
}

```

Search query parameter

```

// Write search query to URL
this.router.navigate(['courses'], { queryParams: { search: this.searchTerm } });

// Read it back on init
const search = this.route.snapshot.queryParamMap.get('search');
if (search) this.searchTerm = search;

Navigating to /courses/2 → CourseDetailComponent displays Algorithms.

```

Typing in search → URL updates to /courses?search=algo
Unknown URL → NotFoundComponent (404 page).

Task 2 — Lazy Loading and Route Guards

Create enrollment feature module

```
ng generate module features/enrollment --routing
# Move EnrollmentFormComponent and ReactiveEnrollmentFormComponent here
```

Lazy loading in app.routes.ts (already shown above)

Chrome DevTools → Network tab:
On initial load: main.js, polyfills.js (no enrollment chunk)
First visit to /enroll: enrollment-module-XXXX.js downloaded on demand

guards/auth.guard.ts

```
import { Injectable } from '@angular/core'; import {
CanActivate, Router } from '@angular/router';
```

```
@Injectable({ providedIn: 'root' }) export
class AuthGuard implements CanActivate {
  isLoggedIn = true; // hardcoded for now
```

```
  constructor(private router: Router) {}
```

```
  canActivate(): boolean {    if
    (this.isLoggedIn) return true;
    this.router.navigate(['/']);
    return false;  }
}
```

guards/unsaved-

changes.guard.ts

```
import { CanDeactivate } from '@angular/router';
import { ReactiveEnrollmentFormComponent } from '../pages/reactive-enrollment-form/...';
```

```
export class UnsavedChangesGuard
  implements CanDeactivate<ReactiveEnrollmentFormComponent> {

  canDeactivate(component: ReactiveEnrollmentFormComponent): boolean {
    if (component.enrollForm.dirty) {
      return window.confirm('You have unsaved changes. Leave?');
    }
    return true;  } }
  isLoggedIn = false → navigating to
  /profile redirects to home page. isLoggedIn = true → /profile
  loads normally.
```

Editing the reactive form and navigating away → browser confirm dialog appears.

Hands-On 8 — HTTP Client: API Integration, Observables & Interceptors

Setup — JSON Server

```
npm install -g json-server
```

```
# db.json
{
  "courses": [
    { "id":1, "name":"Data Structures", "code":"CS101", "credits":4, "gradeStatus":"passed" },
    { "id":2, "name":"Algorithms", "code":"CS102", "credits":3, "gradeStatus":"pending" },
    { "id":3, "name":"Web Development", "code":"CS103", "credits":3, "gradeStatus":"failed" }
  ],
  "students": [],
  "enrollments": [] }
```

```
json-server --watch db.json --port 3000
```

Task 1 — Replace Service Data with HttpClient

app.config.ts — provide HttpClient

```
import { provideHttpClient } from '@angular/common/http';
export const appConfig = {
  providers:
[provideHttpClient(), ...] }; services/course.service.ts
```

— HTTP version

```
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class CourseService {
  private url = 'http://localhost:3000/courses';
  constructor(private http: HttpClient) {}

  getCourses(): Observable<Course[]> { return this.http.get<Course[]>(this.url); }
  getCourseById(id: number): Observable<Course> { return
this.http.get<Course>(`${this.url}/${id}`); }
  createCourse(course: Omit<Course, 'id'>): Observable<Course> {
return this.http.post<Course>(this.url, course); }
  updateCourse(id: number, course: Course): Observable<Course> {
return this.http.put<Course>(`${this.url}/${id}`, course); }
  deleteCourse(id: number): Observable<void> {
return this.http.delete<void>(`${this.url}/${id}`); }
} course-list.component.ts —
```

subscribe

```
ngOnInit() {
this.isLoading = true;
  this.courseService.getCourses().subscribe({ next: courses =>
this.courses = courses, error: err => this.errorMessage =
err.message, complete: () => this.isLoading = false }); } json-
server running → course list loads from http://localhost:3000/courses.
POST /courses → new course appears in db.json and in the UI.
DELETE /courses/2 → course removed from list.
```

Task 2 — RxJS Operators and Error Handling

course.service.ts — operators

```
import { map, catchError, tap, retry, switchMap } from 'rxjs/operators';
import { throwError } from 'rxjs';

getCourses(): Observable<Course[]> { return
this.http.get<Course[]>(this.url).pipe(
  tap(courses => console.log('Courses loaded:', courses.length)), // side effect only
  map(courses => courses.filter(c => c.credits > 0)), // transform retry(2),
  // retry twice on error catchError(err => { console.error(err);
return throwError(() => new Error('Failed to load courses. Please try again.'));
})
);
}

// switchMap — cancels previous inner Observable when courseId changes
selectCourseStudents$(courseId$: Observable<number>) { return
courseId$.pipe(
  switchMap(id => this.enrollmentService.getStudentsByCourse(id))
  // If user selects course 3 before course 2 response arrives, //
course 2 request is cancelled — preventing out-of-order results.
);
}
```

Stopping json-server → after 2 retries, error message shown in template. tap logs "Courses loaded: 3" without modifying the stream. switchMap cancels the previous student-list request on fast course switching.

Task 3 — HTTP Interceptors

Generate interceptors

```
ng generate interceptor interceptors/auth ng
generate interceptor interceptors/error-handler ng
generate interceptor interceptors/loading
```

interceptors/auth.interceptor.ts

```
import { HttpInterceptorFn } from '@angular/common/http';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authReq = req.clone({
    setHeaders: { Authorization: 'Bearer mock-token-12345' }
  });
  return next(authReq);
};
```

interceptors/error-handler.interceptor.ts

```
export const errorHandlerInterceptor: HttpInterceptorFn = (req, next) => {
  return next(req).pipe(
    catchError((err: HttpErrorResponse) => {
      if (err.status === 401) router.navigate(['/login']);
      if (err.status === 500) notificationService.show('Server error!');
      return throwError(() => err);
    })
  );
};
```

interceptors/loading.interceptor.ts

```
import { finalize } from 'rxjs/operators';

export const loadingInterceptor: HttpInterceptorFn = (req, next) => {
  loadingService.setLoading(true);
  return next(req).pipe(
    finalize(() => loadingService.setLoading(false)) //
  );
  // finalize runs on complete OR error — like try/finally
};
```

Register interceptors in app.config.ts

```
provideHttpClient(withInterceptors([
  authInterceptor,
  errorHandlerInterceptor,
  loadingInterceptor ]))
```

DevTools → Network → any API call → Request Headers:
Authorization: Bearer mock-token-12345

Spinner appears on every HTTP call, disappears when complete or on error.
Simulating 401 → navigates to /login.

Hands-On 9 — State Management: NgRx Store, Actions, Reducers & Effects

Setup

```
npm install @ngrx/store @ngrx/effects @ngrx/entity @ngrx/store-devtools
```

Task 1 — NgRx Course State *store/course/course.actions.ts*

```
import { createAction, props } from '@ngrx/store';
import { Course } from '../../models/course.model';

export const loadCourses =
  createAction('[Course] Load Courses');
export const loadCoursesSuccess =
  createAction('[Course] Load Courses Success', props<{ courses: Course[] }>());
export const loadCoursesFailure =
  createAction('[Course] Load Courses Failure', props<{ error: string }>());
```

store/course/course.reducer.ts

```
import { createReducer, on } from '@ngrx/store';
import { loadCourses, loadCoursesSuccess, loadCoursesFailure } from '../course.actions';
import { Course } from '../../models/course.model';

export interface CourseState {
  courses: Course[]; loading:
  boolean; error: string |
  null; }

const initialState: CourseState = { courses: [], loading: false, error: null };

export const courseReducer = createReducer(
  initialState,
  on(loadCourses, state => ({ ...state, loading: true, error: null })),
  on(loadCoursesSuccess, (state, { courses }) => ({ ...state, loading: false, courses })),
  on(loadCoursesFailure, (state, { error }) => ({ ...state, loading: false, error })));
```

store/course/course.selectors.ts

```
import { createFeatureSelector, createSelector } from '@ngrx/store';
import { CourseState } from '../course.reducer';

export const selectCourseState = createFeatureSelector<CourseState>('course');
export const selectAllCourses = createSelector(selectCourseState, s => s.courses);
export const selectCoursesLoading = createSelector(selectCourseState, s => s.loading);
export const selectCoursesError = createSelector(selectCourseState, s => s.error);
```

course-list.component.ts — use Store

```
courses$ = this.store.select(selectAllCourses);

ngOnInit() {
  this.store.dispatch(loadCourses()); // triggers the Effect
}
```

course-list.component.html

```
<app-course-card *ngFor="let c of
courses$ | async"
  [course]="c">
</app-course-card>
```

Redux DevTools timeline:

- [Course] Load Courses (loading: true)
- [Course] Load Courses Success (courses: [...], loading: false)

State tree: { course: { courses:[...], loading:false, error:null } }

Task 2 — NgRx Effects and Enrollment State

store/course/course.effects.ts

```
import { Injectable } from '@angular/core';
```

```
import { Actions, createEffect, ofType } from '@ngrx/effects';
import { switchMap, map, catchError } from 'rxjs/operators';
import { of } from 'rxjs';

@Injectable()
export class CourseEffects {
  loadCourses$ = createEffect(() =>
    this.actions$.pipe(
      ofType(loadCourses),
      switchMap(() =>
        this.courseService.getCourses().pipe(
          map(courses => loadCoursesSuccess({ courses })),
          catchError(err => of(loadCoursesFailure({ error: err.message })))
        )
      )
    )
  );

  constructor(private actions$: Actions,
    private courseService: CourseService) {}
} store/enrollment/enrollment.actions.ts

export const enrollInCourse = createAction('[Enrollment] Enroll', props<{ courseId: number }>());
export const unenrollFromCourse = createAction('[Enrollment] Unenroll', props<{ courseId: number }>());
```

store/enrollment/enrollment.reducer.ts

```
export interface EnrollmentState { enrolledCourseIds: number[]; }
const init: EnrollmentState = { enrolledCourseIds: [] };

export const enrollmentReducer = createReducer(init, on(enrollInCourse, (s,
  { courseId }) => ({ enrolledCourseIds: [...s.enrolledCourseIds, courseId]
})), on(unenrollFromCourse, (s, { courseId }) => ({ enrolledCourseIds:
s.enrolledCourseIds.filter(id => id !== courseId) })))
);
```

Cross-slice selector

```
export const selectEnrolledIds = createSelector(
  selectEnrollmentState, s => s.enrolledCourseIds);

export const selectEnrolledCourses = createSelector(
  selectAllCourses, selectEnrolledIds, (courses, ids) =>
  courses.filter(c => ids.includes(c.id))
); course-card.component.html — enroll
```

toggle

```
<button (click)="toggleEnroll()"> {{ (enrolledIds$ |
  async)?.includes(course.id) ? 'Unenroll' : 'Enroll' }}
</button>
```

Redux DevTools shows full flow:

```
[Course] Load Courses → Effect fires HTTP → [Course] Load Courses Success
[Enrollment] Enroll { courseId: 2 } → state.enrolledCourseIds: [2]
Button label toggles between "Enroll" and "Unenroll" in real time.
```

Hands-On 10 — Unit Testing: Jasmine, Karma & TestBed

Run tests

```
ng test # watch mode
ng test --code-coverage # generates coverage/ folder
```

Task 1 — Testing CourseCardComponent course-

card.component.spec.ts

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { By } from '@angular/platform-browser'; import {
CourseCardComponent } from './course-card.component';

describe('CourseCardComponent', () => {
  let component: CourseCardComponent;
  let fixture: ComponentFixture<CourseCardComponent>;

  const mockCourse = {
    id: 1, name: 'Data Structures', code: 'CS101',
credits: 4, gradeStatus: 'passed' as const  };

  beforeEach(async () => {
    await TestBed.configureTestingModule({
imports: [CourseCardComponent] // standalone
}).compileComponents();

    fixture = TestBed.createComponent(CourseCardComponent);
    component = fixture.componentInstance;  });

  // Test 1 - component creates
  it('should create', () => {
    expect(component).toBeTruthy();  });

  // Test 2 - @Input renders course name
  it('should display the course name', () => {
    component.course = mockCourse;
    fixture.detectChanges(); // triggers change detection - must call before querying DOM
    const h3 = fixture.debugElement.query(By.css('h3')).nativeElement;
    expect(h3.textContent).toContain('Data Structures');  });

  // Test 3 - @Output emits course id on Enroll click
  it('should emit enrollRequested with course id on Enroll click', () => {
    component.course = mockCourse;    fixture.detectChanges();
    spyOn(component.enrollRequested, 'emit');
    fixture.debugElement.query(By.css('button')).nativeElement.click();
    fixture.detectChanges();
    expect(component.enrollRequested.emit).toHaveBeenCalledOnceWith(1);
  });

  // Test 4 - ngOnChanges logs previous/current value
  it('should log ngOnChanges when course input changes', () => {
    spyOn(console, 'log');    component.ngOnChanges({    course:
{
  previousValue: undefined,
currentValue: mockCourse,
firstChange: true,
isFirstChange: () => true    }
  });
    expect(console.log).toHaveBeenCalled();
  });

  // Test 5 - credits pipe via creditLabel
  it('should show correct credit label', () => {
    component.course = mockCourse;
    fixture.detectChanges();
    const creditEl = fixture.debugElement.query(By.css('.credits')).nativeElement;
    expect(creditEl.textContent).toContain('4 Credits');  }); }); ng test output:
SUMMARY:
  5 specs, 0 failures
✓ CourseCardComponent should create
✓ CourseCardComponent should display the course name
```

- ✓ CourseCardComponent should emit enrollRequested with course id on Enroll click
- ✓ CourseCardComponent should log ngOnChanges when course input changes
- ✓ CourseCardComponent should show correct credit label

Task 2 — Testing CourseService and NgRx Store Component

course.service.spec.ts

```
import { TestBed } from '@angular/core/testing'; import {
HttpClientTestingModule, HttpTestingController } from
'@angular/common/http/testing'; import { CourseService } from
'./course.service';

describe('CourseService', () => { let
service: CourseService; let
httpMock: HttpTestingController;

const mockCourses = [ { id:1, name:'Data Structures', code:'CS101', credits:4,
gradeStatus:'passed' }, { id:2, name:'Algorithms', code:'CS102', credits:3,
gradeStatus:'pending' } ];

beforeEach(() => {
TestBed.configureTestingModule({
imports: [HttpClientTestingModule],
providers: [CourseService] });
service = TestBed.inject(CourseService);
httpMock = TestBed.inject(HttpTestingController);
});

afterEach(() => httpMock.verify()); // no outstanding requests

// Test 1 - GET courses
it('should fetch courses via GET', () => {
service.getCourses().subscribe(courses => {
expect(courses.length).toBe(2);
expect(courses[0].name).toBe('Data Structures');
});
const req = httpMock.expectOne('http://localhost:3000/courses');
expect(req.request.method).toBe('GET'); req.flush(mockCourses);
});

// Test 2 - error handling
it('should handle 500 error and return custom message', () => {
service.getCourses().subscribe({ next: () =>
fail('should have errored'),
error: (err) => expect(err.message).toContain('Failed to load courses')
});
httpMock.expectOne('http://localhost:3000/courses').flush('Server
error', { status: 500, statusText: 'Internal Server Error' });
});
}); course-list.component.spec.ts —
```

MockStore

```
import { provideMockStore, MockStore } from '@ngrx/store/testing';
import { selectAllCourses } from '../store/course/course.selectors';

describe('CourseListComponent with NgRx', () => {
let store: MockStore;
let fixture: ComponentFixture<CourseListComponent>;

const mockCourses = [ { id:1, name:'Data Structures', code:'CS101',
credits:4, gradeStatus:'passed' } ];

beforeEach(async () => {
```

```

    await TestBed.configureTestingModule({
      imports: [CourseListComponent],
      providers: [
        provideMockStore({
          initialState: { course: { courses: mockCourses, loading: false, error: null } }
        })
      ]
    }).compileComponents();

    store = TestBed.inject(MockStore);
    fixture = TestBed.createComponent(CourseListComponent);
    fixture.detectChanges();

    it('should render one course card', () => {
      const cards = fixture.debugElement.queryAll(By.css('app-course-card'));
      expect(cards.length).toBe(1);
    });

    it('should show loading indicator when loading is true', () => {
      store.setState({ course: { courses: [], loading: true, error: null } });
      fixture.detectChanges();
      const loader = fixture.debugElement.query(By.css('.loading'));
      expect(loader).toBeTruthy();
    });
  });
});
ng test output: SUMMARY:
  4 specs, 0 failures
CourseService should fetch courses via GET
CourseService should handle 500 error and return custom message
CourseListComponent with NgRx should render one course card
CourseListComponent with NgRx should show loading indicator when loading is true

```